



Streamlit Avançado

MÓDULO 02 - SEMANA 11



AGENDA DA SEMANA

Terça-feira — Conexão a Dados & Segurança (secrets.toml e PostgreSQL)

- Integração de banco de dados relacional com `st.connection("postgresql", type="sql")`[cite: 1, 2].
- Leitura e execução de consultas SQL diretamente para DataFrames com Pandas[cite: 1, 2].
- Gestão de credenciais e segurança com `.streamlit/secrets.toml` e fallback para arquivos locais (CSV).

Quinta-feira — Interatividade Dinâmica, Filtros Cruzados & Cache Avançado

- Construção de filtros reativos com atualização de gráficos e KPIs em tempo real.
- Otimização de performance: diferença entre `@st.cache_data` (DataFrames/consultas) e `@st.cache_resource` (conexões de banco)[cite: 1, 2].
- Parâmetros avançados de cache (ttl para expiração automática e mensagens de *loading*).
- Gerenciamento de estado da aplicação com `st.session_state`.

Sexta-feira — Deploy na Nuvem (Streamlit Sharing) & Projeto Final

- Preparação do ambiente de produção e geração do `requirements.txt`.
- Versionamento no GitHub e integração com o Streamlit Community Cloud.
- Configuração de *Secrets* no painel da nuvem e publicação da URL pública.
- **Desafio Prático:** Publicação ao vivo do Dashboard integrado com banco de dados e filtros reativos.

DRIVE DA SEMANA E GIT

Google Drive da semana:

<https://drive.google.com/drive/folders/19qpal8N-swkr1Q8ZWMqWnlbtWUj8SCq1?usp=sharing>

GitHub da semana:

https://github.com/Visualizacao-Dados-BI-T2/M2_Semana_11.git

AULA 1

CONEXÃO A BANCOS DE DADOS, PANDAS E SEGURANÇA COM
SECRETS.TOML

POR QUE CONECTAR A UM BANCO DE DADOS RELACIONAL?

- Conectando a Bancos de Dados (st.connection): Como o Streamlit gerencia conexões SQL nativamente com `st.connection("postgresql", type="sql")`[cite: 1, 2].
- Segurança & Boas Práticas (secrets.toml): Armazenamento seguro de credenciais no arquivo `.streamlit/secrets.toml` e inclusão no `.gitignore`.
- Leitura com Pandas & Fallback Local: Execução de `st.connection.query()` e tratativa para carregar base em CSV caso a conexão remota falhe[cite: 1, 2].
- Prática Guiada: Criação de queries dinâmicas (WHERE, GROUP BY) exibindo resultados na tela.

O GERENCIADOR NATIVO `st.connection()`

- **Abstração Simplificada:** O Streamlit introduziu o `st.connection()` para gerenciar conexões com bancos SQL sem a necessidade de instanciar pools complexos manualmente[cite: 1, 2].
 - **Sintaxe Básica:**
`conn = st.connection("postgresql", type="sql")`
- **Vantagens:** Gerencia conexões ativas de forma eficiente, integrando diretamente com SQLAlchemy e Pandas[cite: 1, 2].

INSTALAÇÃO DOS DRIVERS DE BANCO NO PYTHON

- **Dependências Necessárias:** Além do streamlit e pandas, precisamos dos conectores do PostgreSQL.
 - **Comando de Instalação via Terminal:**
Bash
pip install psycopg2-binary sqlalchemy

INSTALAÇÃO DOS DRIVERS DE BANCO NO PYTHON

- **Dependências Necessárias:** Além do streamlit e pandas, precisamos dos conectores do PostgreSQL.
 - **Comando de Instalação via Terminal:**
Bash
pip install psycopg2-binary sqlalchemy
- **O papel de cada pacote:**
 - *SQLAlchemy: Biblioteca que gerencia as conexões e abstrai os comandos SQL no Python.*
 - *psycopg2-binary: Driver de baixo nível para comunicação do Python com o PostgreSQL.*

SEGURANÇA EM PRIMEIRA LUGAR: O RISCO DE CREDENCIAIS EXPOSTAS

- **Má Prática:** Chumbamento de senhas (*hardcoding*) diretamente no script `app.py`.
- **O Perigo:** Se o código for enviado para um repositório público no GitHub, as senhas e portas de acesso do banco de dados ficam totalmente expostas.
- **Boa Prática:** Isolar parâmetros sensíveis (Host, Usuário, Senha, Porta, Database) em arquivos de configuração locais omitidos do versionamento.

GERENCIAMENTO DE CREDENCIAIS COM secrets.toml

- **Estrutura de Pastas:**
 - meu_projeto/
 - |—— .streamlit/
 - |—— secrets.toml # Arquivo de senhas (NÃO vai para o GitHub)
 - |—— .gitignore # Especifica o que o Git deve ignorar
 - |—— app.py # Script da aplicação
 - |—— dados.csv # Base local para fallback
- **A regra de ouro:** Adicionar .streamlit/secrets.toml dentro do arquivo .gitignore para prevenir vazamentos de dados em commits.

ESTRUTURA PRÁTICA DO secrets.toml

- **Formato TOML:** O Streamlit mapeia automaticamente este arquivo para o dicionário `st.secrets`.
 - **Exemplo de Conteúdo (`.streamlit/secrets.toml`):**
 - `[connections.postgresql]`
 - `dialect = "postgresql"`
 - `host = "seu-servidor-postgres.com"`
 - `port = 5432`
 - `database = "nome_do_banco"`
 - `username = "seu_usuario"`
 - `password = "sua_senha_secreta"`

EXECUTANDO QUERIES SQL COM `st.connection.query()`

- **Retorno Direto em DataFrame:** O método `.query()` executa a instrução SQL e devolve um DataFrame Pandas limpo e pronto para uso[cite: 1, 2].
 - **Exemplo de Código:**
 - `import streamlit as st`
 -
 - `conn = st.connection("postgresql", type="sql")`
 -
 - `# Executa a query e retorna um DataFrame do Pandas`
 - `df = conn.query("SELECT * FROM vendas WHERE status = 'Aprovado';", ttl="10m")`
 -
 - `st.dataframe(df)`

TRATAMENTO DE ERROS E FALLBACK LOCAL (CSV)

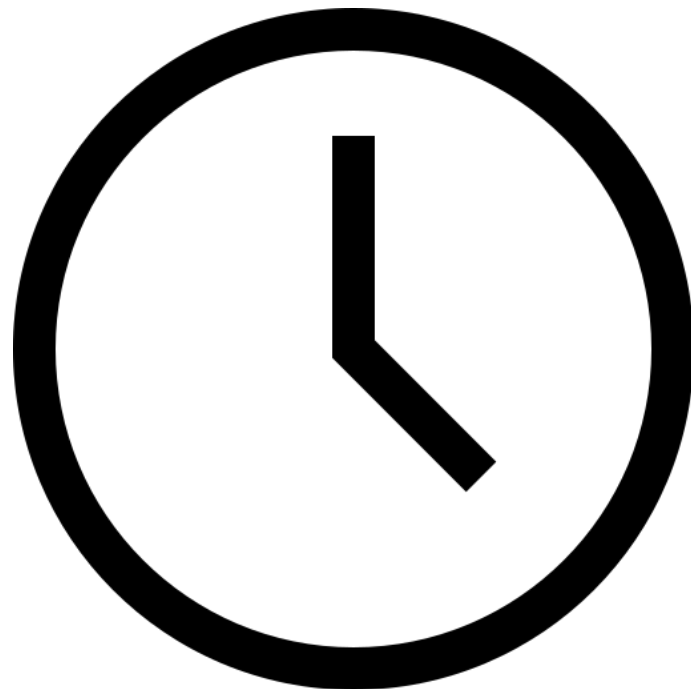
- **Garantindo Resiliência:** Caso a conexão com o banco remoto falhe (queda de rede, credenciais inválidas), a aplicação não deve quebrar.
 - **Estrutura com try/except:**
 - **try:**
 - `conn = st.connection("postgresql", type="sql")`
 - `df = conn.query("SELECT * FROM vendas;", ttl="10m")`
 - `st.success("Dados carregados do banco PostgreSQL!")`
 - **except Exception as e:**
 - `st.warning("Falha na conexão com o banco. Carregando dados locais em CSV...")`
 - `df = pd.read_csv("dados.csv")`

INTERVALO!

Finalizamos o nosso primeiro período de hoje. Que tal descansar um pouco?!

Nos vemos em 15 minutos.

Início: 20:30
Retorno: 20:45



PRÁTICA: CONSULTA DINÂMICA AO BANCO

Objetivo: Construir uma tela onde o usuário seleciona a Categoria e o aplicativo roda a query SQL trazendo apenas o recorte filtrado.

Passo a Passo:

1. Criar st.selectbox com as categorias.
2. Injetar o parâmetro selecionado na consulta via SQL parametrizado (params).
3. Exibir a tabela e um cartão de KPI com o volume total de vendas da categoria.

QUESTIONÁRIO DE ENTENDIMENTO DA TURMA

Objetivo: Construir uma tela onde o usuário seleciona a Categoria e o aplicativo roda a query SQL trazendo apenas o recorte filtrado.

Passo a Passo:

1. Criar st.selectbox com as categorias.
2. Injetar o parâmetro selecionado na consulta via SQL parametrizado (params).
3. Exibir a tabela e um cartão de KPI com o volume total de vendas da categoria.

AULA 2

REATIVIDADE DINÂMICA, FILTROS CRUZADOS E CACHE AVANÇADO

O FLUXO DE EXECUÇÃO SOB A ÓTICA DE BANCOS DE DADOS

- **Revisão do Ciclo de Reexecução (*Rerun Model*):** Qualquer interação do usuário na tela (mover slider, alterar selectbox) faz o Streamlit reexecutar todo o script Python do topo ao fim.
- **O Novo Gargalo:** Em aplicações conectadas a bancos de dados, cada reexecução sem otimização resulta em uma nova requisição SQL no servidor de banco de dados.
- **Impacto:** Consultas repetitivas tornam a aplicação extremamente lenta, travam a interface e sobrecarregam o servidor do PostgreSQL.

CONSTRUÇÃO DE FILTROS REATIVOS CRUZADOS

- **Interatividade em Cadeia:** Permitir que seleções feitas em um filtro afetem diretamente os resultados e opções dos demais componentes na tela.
- **Filtros Simultâneos:** Combinação de `st.multiselect` (Categorias), `st.date_input` (Período de Vendas) e `st.slider` (Faixa de Valor).
- **Aplicação no DataFrame:**
 - *# Aplicação encadeada de filtros no Pandas*
 - `df_filtrado = df[`
 - `(df["categoria"].isin(categorias_selecionadas)) &`
 - `(df["data"] >= data_inicio) &`
 - `(df["data"] <= data_fim)`
 - `]`

O DESAFIO DE PERFORMANCE E CONSULTAS REPETITIVAS

- **Problema Comum:** Sem uma camada de cache, o comando `conn.query()` buscará as mesmas linhas no PostgreSQL repetidas vezes a cada segundo.
- **Experiência do Usuário (UX):** A tela exibe congelamentos (*lag*) e tempo de espera excessivo a cada simples clique do usuário.
- **A Solução:** Mecanismo avançado de Caching do Streamlit para armazenar na memória do servidor o resultado de funções e instâncias de banco de dados.

GERENCIAMENTO DE MEMÓRIA: `@st.cache_data`

- **Destino Principal:** Utilizado para armazenar dados mutáveis, serializáveis e computados (DataFrames do Pandas, resultados de queries SQL e retornos de APIs).
- **Funcionamento Interno:** Salva o resultado retornado pela função e cria cópias seguras para evitar mutações de dados indesejadas entre interações do usuário.
- **Exemplo:**
 - `@st.cache_data`
 - `def buscar_vendas(query_sql):`
 - `return conn.query(query_sql)`

GERENCIAMENTO DE MEMÓRIA: `@st.cache_resource`

- **Destino Principal:** Utilizado para armazenar objetos globais não serializáveis e conexões brutas (conexão com banco de dados PostgreSQL, modelos de Machine Learning e clientes de API).
- **Funcionamento Interno:** Mantém e compartilha exatamente a mesma instância do objeto em memória para todas as sessões e usuários conectados na aplicação.
- **Exemplo:**
 - `@st.cache_resource`
 - `def conectar_banco():`
 - `return st.connection("postgresql", type="sql")`

QUADRO COMPARATIVO: `cache_data` VS `cache_resource`

@st.cache_data:

- **Uso:** DataFrames, listas, dicionários, tabelas e resultados SQL.
- **Comportamento:** Retorna uma nova cópia do dado para cada execução.
- **Exemplo:** Resultado da consulta `SELECT * FROM vendas`.

@st.cache_resource:

- **Uso:** Conexões de Banco de Dados, Sessões do SQLAlchemy e Modelos de IA.
- **Comportamento:** Retorna a própria referência do objeto original na memória.
- **Exemplo:** Instância do conector `st.connection()`.

ESTRATÉGIAS AVANÇADAS: TEMPO DE EXPIRAÇÃO (ttl)

- **O Parâmetro ttl (*Time To Live*):** Define por quanto tempo o resultado de uma consulta ficará armazenado no cache antes de ser automaticamente destruído e reconsultado no banco.
- **Uso Recomendado:** Essencial para aplicações corporativas onde os dados do PostgreSQL são atualizados de forma constante no banco.
- **Exemplo Prático:**
 - *# O cache expira automaticamente a cada 10 minutos (600 segundos)*
 - *df = conn.query("SELECT * FROM vendas;", ttl=600)*

MELHORANDO A UX COM MENSAGENS DE CARREGAMENTO

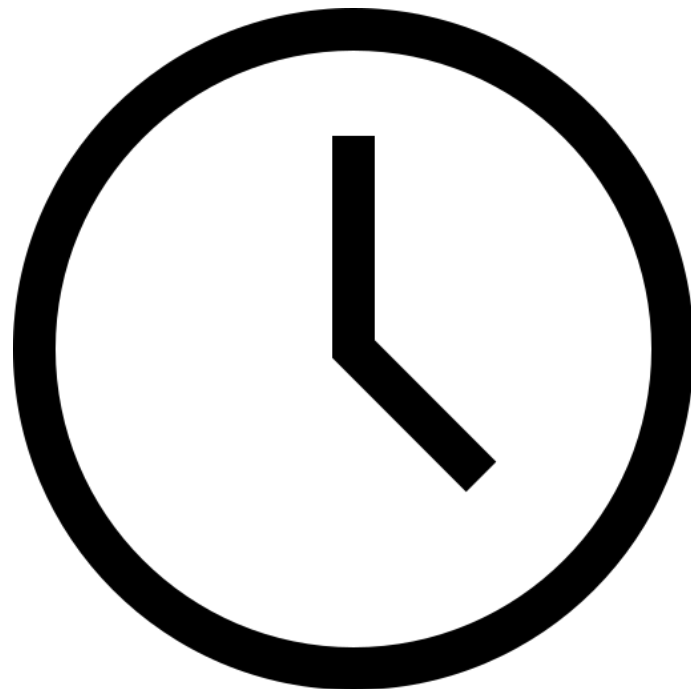
- **Evitando Telas Brancas:** Informar o usuário que um processamento ou consulta SQL pesada está acontecendo em segundo plano.
- **O Parâmetro `show_spinner`:**
 - `@st.cache_data(show_spinner="Buscando dados no PostgreSQL...", ttl=300)`
 - `def carregar_relatorio():`
 - `return conn.query("SELECT * FROM relatorio_consolidado;")`
- **Controle Manual com `st.spinner`:**
 - `with st.spinner("Atualizando indicadores..."):`
 - `# Código do processamento pesado`
 - `pass`

INTERVALO!

Finalizamos o nosso primeiro período de hoje. Que tal descansar um pouco?!

Nos vemos em 15 minutos.

Início: 20:30
Retorno: 20:45



PERSISTÊNCIA DE ESTADO COM `st.session_state`

- **O Problema da Perda de Variáveis:** Como o script roda do início ao fim a cada clique, variáveis locais convencionais são resetadas a todo momento.
- **O que é o `st.session_state`?** Um dicionário de dados persistente associado à sessão individual do navegador do usuário no Streamlit.
- **Aplicações:** Guardar seleções anteriores, gerenciar fluxos com botões de etapa/página e armazenar parâmetros de preferências do usuário.

EXEMPLO PRÁTICO: BOTÃO PARA RESETAR FILTROS

- **Uso Combinado do Estado de Sessão e Callbacks:**

- `import streamlit as st`
- `# Inicializa o estado caso não exista`
- `if "categoria_selecionada" not in st.session_state:`
- `st.session_state.categoria_selecionada = "Todas"`
- `def resetar_filtros():`
- `st.session_state.categoria_selecionada = "Todas"`
- `# Widget vinculado ao session_state`
- `st.selectbox(`
- `"Selecione a Categoria",`
- `["Todas", "Eletrônicos", "Roupas"],`
- `key="categoria_selecionada"`
- `)`
- `# Botão de Reset acionando a função Callback`
- `st.button("Resetar Filtros", on_click=resetar_filtros)`

QUESTIONÁRIO DE ENTENDIMENTO DA TURMA

-

AULA 3

DEPLOY EM PRODUÇÃO, STREAMLIT CLOUD

PREPARANDO A APLICAÇÃO PARA O AMBIENTE DE PRODUÇÃO

- **Do Local para o Servidor:** Em ambiente local, as bibliotecas estão salvas no seu computador; na nuvem, o servidor precisa saber exatamente o que instalar para rodar o app.
- **O Arquivo requirements.txt:** Especifica a lista de dependências e bibliotecas Python necessárias para a execução da aplicação.
- **Geração Automática do requirements.txt:**
 - Uso do pacote pipreqs ou criação manual apontando as versões das bibliotecas.
 - Conteúdo indispensável: streamlit, pandas, plotly, psycpg2-binary, sqlalchemy.

ESTRUTURA DO REPOSITÓRIO NO GITHUB

- **Organização para Deploy:** O repositório precisa estar estruturado de forma limpa para ser lido corretamente pelos servidores de nuvem.
- **Estrutura Recomendada no GitHub:**
 - **meu-dashboard-streamlit/**
 - |—— `.streamlit/`
 - | |—— `config.toml` *# Configurações visuais do app (opcional)*
 - |—— `.gitignore` *# Ignora o .venv e o secrets.toml*
 - |—— `app.py` *# Script principal da aplicação*
 - |—— `dados.csv` *# Base local para fallback*
 - |—— `requirements.txt` *# Lista de dependências Python*
- **Lembrete de Segurança:** Garantir que o arquivo `.streamlit/secrets.toml` **NÃO** seja commitado no GitHub!

O QUE É O STREAMLIT COMMUNITY CLOUD?

- **Plataforma Oficial de Hospedagem:** Serviço gratuito oferecido pela própria equipe do Streamlit para publicar Data Apps diretamente a partir do GitHub.
Escopo - Visualização de Dados e BI [T1 e T2] - M2S07 - Mini-Projeto Avaliativo.pdf
- **Vantagens Principais:**
 - Integração nativa e contínua com o repositório no GitHub (Deploy automático a cada git push).
 - Suporte nativo ao gerenciamento de **Secrets** e variáveis de ambiente em nuvem.
 - Criação de link público acessível por qualquer pessoa ou cliente (<https://seu-app.streamlit.app>).

PASSO A PASSO DO DEPLOY 1: CONECTANDO O REPOSITÓRIO

- **Passo 1:** Criar uma conta no share.streamlit.io vinculada ao seu perfil do **GitHub**.
- **Passo 2:** No painel do Streamlit Cloud, clicar no botão "**New app**".
- **Passo 3:** Selecionar o repositório do projeto, a branch (ex: main) e o arquivo de entrada principal (ex: app.py).

ASSO A PASSO DO DEPLOY 2: CONFIGURANDO AS SECRETS EM NUVEM

- **Substituindo o secrets.toml Local:** Como o arquivo de senhas local não vai para o Git, ele deve ser cadastrado no painel web do Streamlit Cloud.
- **Acessando as Configurações:** Em *Advanced Settings* -> *Secrets*, colar exatamente a mesma estrutura TOML criada na Aula 1:
 - `[connections.postgresql]`
 - `dialect = "postgresql"`
 - `host = "seu-servidor-postgres.com"`
 - `port = 5432`
 - `database = "nome_do_banco"`
 - `username = "seu_usuario"`
 - `password = "sua_senha_secreta"`

VISÃO GERAL DE OUTRAS ALTERNATIVAS DE HOSPEDAGEM

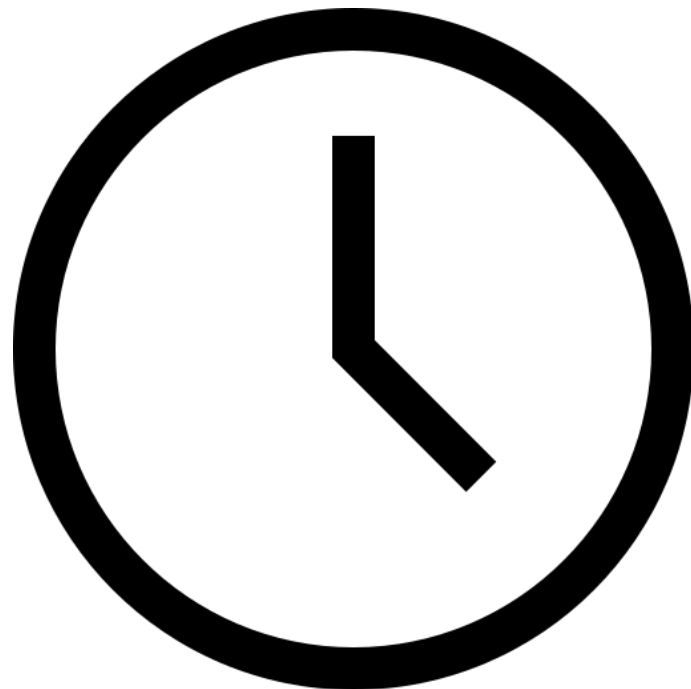
- **Panorama de Mercado para Aplicações em Python:**
 - **Hugging Face Spaces:** Excelente alternativa gratuita para deploys de Streamlit e Gradio com foco em IA/Data Science.
 - **Render / Railway:** Plataformas de *Platform as a Service* (PaaS) robustas que substituíram o Heroku no mercado.
 - **Docker + AWS/GCP:** Abordagem corporativa avançada para containerizar a aplicação Streamlit e rodar em instâncias de nuvem privada.

INTERVALO!

Finalizamos o nosso primeiro período de hoje. Que tal descansar um pouco?!

Nos vemos em 15 minutos.

Início: 20:30
Retorno: 20:45



DESAFIO PRÁTICO FINAL: PUBLICANDO O DASHBOARD COMPLETO

Cenário: Construção e publicação oficial do Dashboard Comercial com dados integrados e performance otimizada.

Requisitos do Aplicativo em Produção:

1. Conexão resiliente ao PostgreSQL com fallback para CSV local.
2. Painel lateral com filtros cruzados reativos (`st.sidebar`).
3. Consultas e DataFrames otimizados com `@st.cache_data` e `ttl`.
4. Visualização interativa em Plotly e indicadores de KPI em colunas (`st.columns` e `st.metric`).
5. Deploy concluído no Streamlit Cloud com link público gerado.

ROTEIRO DE DESENVOLVIMENTO DO DESAFIO

1. **Passo 1:** Testar a conexão local e garantir que o secrets.toml funciona corretamente.
Escopo - Visualização de Dados e BI [T1 e T2] - M2S07 - Mini-Projeto Avaliativo.pdf
2. **Passo 2:** Aplicar as regras de cache (@st.cache_data / @st.cache_resource) nas funções de leitura.
Visualização de Dados e Business Intelligence T2 - M2S10 - Aulas - Savio+ 1
3. **Passo 3:** Criar o arquivo requirements.txt na raiz do projeto.
Escopo - Visualização de Dados e BI [T1 e T2] - M2S07 - Mini-Projeto Avaliativo.pdf
4. **Passo 4:** Subir o código para o repositório público do GitHub.
Escopo - Visualização de Dados e BI [T1 e T2] - M2S07 - Mini-Projeto Avaliativo.pdf
5. **Passo 5:** Fazer o deploy no Streamlit Cloud, cadastrar as Secrets no painel e validar o link gerado.

ROTEIRO DE DESENVOLVIMENTO DO DESAFIO

1. **Passo 1:** Testar a conexão local e garantir que o `secrets.toml` funciona corretamente.
2. **Passo 2:** Aplicar as regras de cache (`@st.cache_data` / `@st.cache_resource`) nas funções de leitura.
3. **Passo 3:** Criar o arquivo `requirements.txt` na raiz do projeto.
4. **Passo 4:** Subir o código para o repositório público do GitHub.
5. **Passo 5:** Fazer o deploy no Streamlit Cloud, cadastrar as Secrets no painel e validar o link gerado.

QUESTIONÁRIO DE ENTENDIMENTO DA TURMA

1.

AGENDA

- Informações do template
- Exemplo com texto (Light mode)
- Exemplo com texto (Dark mode)
- Exemplo com imagens (Light mode)
- Exemplo com imagens (Dark mode)

INFORMAÇÕES DO TEMPLATE

- Título da Aula:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 32
 - Cor: Branco
- Sub-título da Aula:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 16
 - Cor: Verde
- Título do Slide:
 - Fonte: Roboto
 - Formato: Maiúsculo
 - Tamanho: 24
 - Cores: Verde
- Parágrafos:
 - Fonte: Roboto
 - Tamanho: 12 a 18
 - Cores: Branco (Modo Escuro) ou Preto (Modo Claro)
- Marcadores de tópicos:
 - Formatos: Símbolos ou Alfanuméricos
 - Cor: Verde

EXEMPLO COM TEXTO (MODO CLARO)

O NOMEPRODUTO, é uma formação imersiva do LAB 365, alinhada com as mais recentes tecnologias de mercado, com o objetivo de acelerar a carreira de NOMEAREA.

EXEMPLO COM TEXTO (MODO ESCURO)

O NOMEPRODUTO, é uma formação imersiva do LAB 365, alinhada com as mais recentes tecnologias de mercado, com o objetivo de acelerar a carreira de NOMEAREA.

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique [aqui](#) ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.